It is a common practice to express the duty cycle value using a variable instead of a constant value. One way that we could do this would be to use a `for` loop, as in the following code sample:

```
for (int i=0; i<=255; i+=5) {
  analogWrite(5, i);
  delay(20);
}
```

This sample code uses a `for` loop to increment a counter, `i`, in steps of five beginning with 0 and ending with 255. This value expressed as a variable is then output to PWM pin 5 using `analogWrite()` followed by a brief 20-millisecond delay. If connected to an LED, this would have the effect of starting with the LED off and gradually increasing brightness until the LED was at full brightness. Likewise, if we used our fan circuit, the fan would start at off and increase in speed until it is spinning at full speed.

We could even combine analog output with analog input, as in the following simple sketch in Listing 6-3.

Listing 6-3. *analogWrite() Example*

```
int analogValue = 0;
int ledPin = 5;

void setup() {
}

void loop() {
  analogValue = analogRead(A0) / 4;
  analogWrite(ledPin, analogValue);
  delay(20);
}
```

This simple little sketch is the most basic form of our source code for the project, although if you try it you'll see it doesn't work quite as well. This code simply takes an analog reading and routes that value to an analog output. Again, there are no statements needed in the `setup()` function because we are using only analog functions.

One challenge that we face when using a value from an analog input and sending it to an analog output, is that the `analogRead()` function returns a value in the range of 0 to 1023, while `analogWrite()` only works with values in the range of 0 to 255. To get around this, when assigning the value to the variable `analogValue` in the first line of this most recent `loop()` function, we take the reading from analog pin A0 and divide this by 4 before assigning to our variable. All should work fine because 1024, the total number of possible values in a 10-bit reading, divided by 4 will result in 255, which is the maximum number of values that we can use in the 8-bit PWM function.

Now depending on the sensor we are using on our analog input, this may not result in either the most accurate or the prettiest results. To fix this, we will need to make use of some techniques and functions, some of which were used in our project earlier, to find out what our readings actually are and increase their accuracy, beginning with the next analog function and proceeding on through the rest of the chapter.

analogReference()

By using the `analogRead()` function, the Arduino assumes a voltage range of between 0v and +5v. Increasingly though, more and more sensors are using lower operating voltages from +3.3v to as low as